

Veer D1 High Level Design

Course: Computer Science Senior Design I

Advisor: Dr. Jillian Aurisano

Team: Grady Algire, Brady Cooper, Briar Elliot, Dominic Rowland, Aiden Ward

Design version: 1.1 | October 8, 2026

Project goal: Convert driver speech into routing constraints for hands-free navigation

D1 purpose: D1 opens the two D0 components whose internal behavior is most important to the first release: C2 Speech Understanding and C3 Route Planning. The remaining C1 Mobile Navigation and C4 Usage Monitor are deferred to D2

Traceability basis: US-01, US-02, US-03, US-05; AC-01.1 through AC-01.4; D04, D05, D07, D08

1. Header, Scope, and Conventions

This D1 document details C2 Speech Understanding and C3 Route Planning from the D0 design. C1 Mobile Navigation and C4 Usage Monitor are deferred to D2. The detailed design preserves the D0 responsibilities: C2 converts transient speech into proposed typed routing constraints, while C3 produces route proposals that satisfy validated trip constraints.

Conventions. Team-built components are labeled C; external dependencies are labeled E. Rectangles in the data model represent entities/records. PK identifies a primary key and FK identifies a foreign key. Relationships use 1 and * at the ends to show cardinality. Solid relationship lines represent ownership or containment; dashed conceptual links represent data used for evaluation without persistence. Interface arrows indicate request direction; responses travel back over the same interface. Types follow the D0 conventions: string = UTF-8, integer = whole number, number = finite decimal, boolean = true/false, instant = ISO 8601 UTC, [] = array, ? = optional. Durations are seconds, distances are meters, and precipitation probability is 0–1.

Requirement trace. US-01 requires a food stop with a detour no greater than 10 minutes; US-02 requires an avoid-rain preference; US-05 requires explicit spoken confirmation before changing a route. AC-01.1 requires the approved route to become active within 3 seconds after the driver's approval utterance ends. AC-01.3 and AC-01.4 require bounded recovery behavior for no-match and unparseable speech.

2. Data Model (the D1 diagram)

D1 Core Data Model – C2/C3

PK = primary key. FK = foreign key where persisted. Cardinality uses 1 and * notation

Transient provider records are modeled for behavior when they are not retained after the trip.

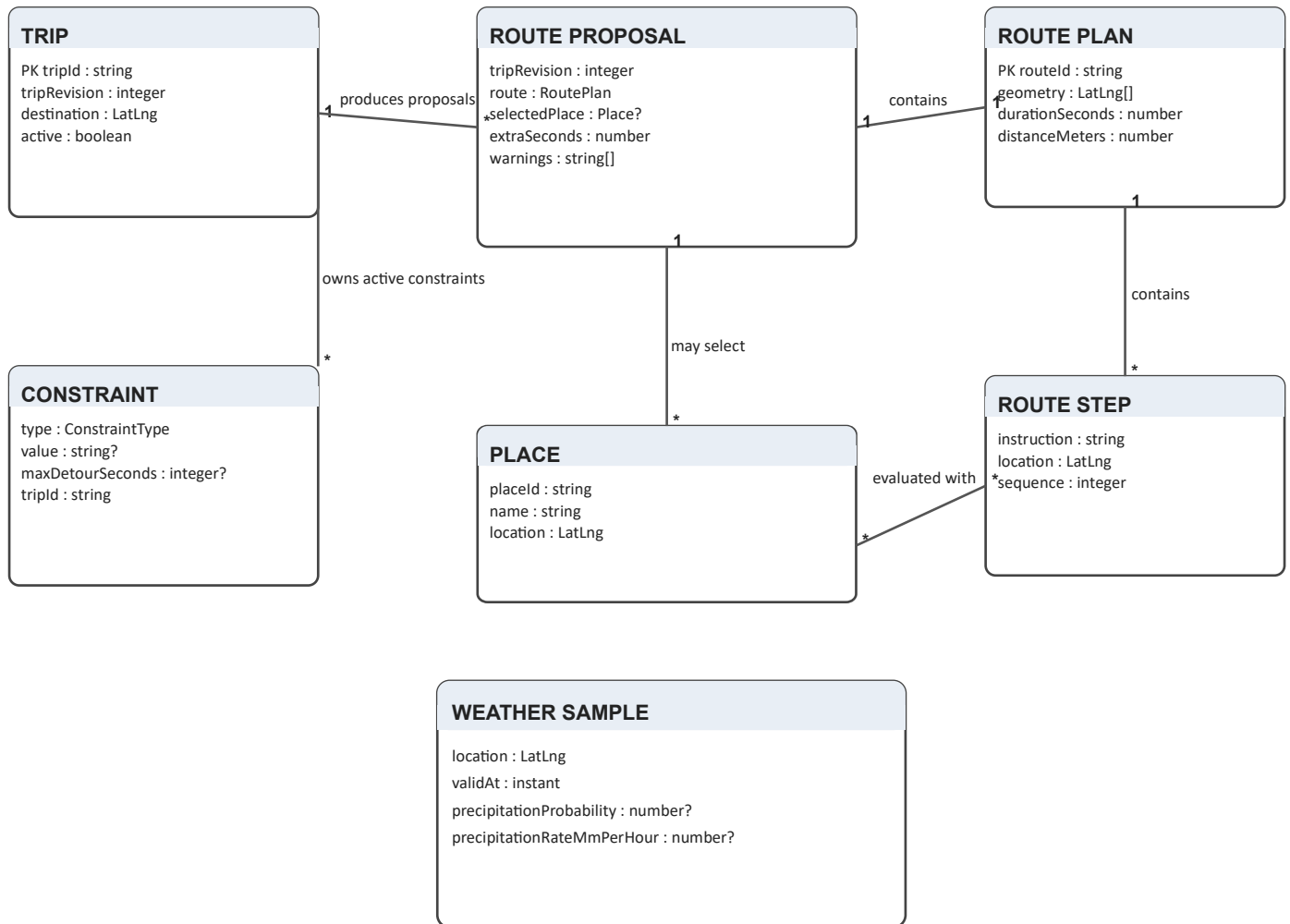


Figure 1. Core C2/C3 data model.

The C2/C3 data model centers on the trip-scoped routing state. A Trip owns the active Constraints used for a replan. C3 produces Route Proposals containing a candidate Route Plan and, when applicable, a selected Place. Route Plans contain Route Steps. Weather Sample records are obtained from E4 and are evaluated against route locations and travel times; they are not retained after the trip.

Structural decisions. The model deliberately distinguishes entities from attributes. A Trip is an entity because it establishes the lifetime and revision boundary for active constraints and route state. A Constraint is an entity/record because multiple independent constraints may be active at once and each has its own type and optional value. Route Proposal and Route Plan are separate because a proposal is a decision about a route, while a Route Plan is the route data itself. Route Step is a repeating component of a route and therefore is represented separately rather than as a single string. Place is separate because a selected business/category match has identity and location. Weather Sample remains a transient record rather than a persisted entity because D0 prohibits retaining trip conversation/location data as part of C4 and the first release does not require a weather history.

Decision	Choice and justification
Trip → Constraint	1-to-many. A trip may contain zero or many active constraints; each active constraint belongs to one trip. This supports US-01/US-02 and clears all active constraints at trip end.

Decision	Choice and justification
Trip → Route Proposal	1-to-many conceptually, but proposals are transient. Multiple candidate proposals can be generated for one trip revision without replacing the active route until explicit approval.
Route Plan → Route Step	1-to-many. A route consists of an ordered sequence of navigation steps. The sequence is behaviorally significant even though the D0 type lists steps as an array.
Route Proposal → Place	0-to-1. A proposal may contain a selected place for add-waypoint requests, but highway/toll/weather requests do not require one.
Indexes	If a persistent implementation is used, index Trip.tripId and tripRevision for fast stale-response checks, and index UsageEvent.tripId/eventId for C4 deduplication. No index is justified for transient provider records.
Storage model	Relational storage is preferred for retained usage metadata because trips, provider events, and summaries have stable relationships and require reliable identifiers. C2/C3 routing state itself remains primarily in memory and is cleared at trip end.

3. Core Algorithms

Two computations are treated as product-critical: (1) C2 validation of provider interpretation into a supported routing constraint and (2) C3 constraint-aware route selection. The first prevents an AI response from directly controlling routing; the second determines whether Veer actually satisfies the driver's request.

3.1 Speech Interpretation Validation

Problem solved. C2 receives transcription, intent, confidence, and an optional Constraint from E2. The validation algorithm determines whether the provider output is structurally valid, uses a supported intent and constraint type, and contains values that can safely be passed to C3. Invalid or unclear results become an error/unknown result and leave the current route unchanged.

Inputs. `transcript: string; intent: enum(propose, accept, reject, unknown); constraint: Constraint?; confidence: number; allowedConstraintTypes: string[]`. **Output.** A validated `Constraint?`, `intent`, `confidence`, or an `Error`.

Expected complexity. Validation is $O(k)$, where k is the number of fields and allowed enum values checked; k is bounded and very small for Veer's fixed contract. At $100\times$ the number of trip requests, the per-request validation cost remains effectively constant. The expensive work is external transcription/interpretation, not this validation pass.

Why this approach. A fixed allow-list and typed validation is preferred over free-form routing instructions because D04 explicitly represents speech as typed routing constraints. It also prevents the AI provider from directly mutating the route and provides deterministic failure behavior.

Edge cases. Empty transcript → unknown/repeat prompt; missing constraint for a propose intent → invalid response; unsupported constraint type → reject; confidence outside 0–1 → reject; missing required waypoint value → reject; accept/reject without a corresponding confirmation context → do not change the route; provider timeout/quota → retry or recovery according to the `Error.retryable` field.

3.2 Constraint-Aware Route Selection

Problem solved. C3 must turn validated constraints into a safe route proposal. It requests candidate routes/places from E3, obtains relevant weather data from E4 when precipitation avoidance is active, filters candidates that violate hard limits, then ranks the remaining candidates using the user's stated preference and detour cost. The current route is never replaced by C3 directly.

Inputs. `location: LocationFix; destination: LatLng; currentRoute: RoutePlan?; constraints: Constraint[]; tripRevision: integer.` **Output.** `RouteProposal? or Error.`

Selection procedure. First normalize the active constraints and determine whether a waypoint search, highway/toll avoidance, or weather evaluation is needed. Request alternate routes and/or places from E3. For an add-waypoint request, discard places whose resulting detour exceeds 600 seconds. For highway/toll preferences, rank candidates that satisfy the requested avoidance flags ahead of candidates that do not. For precipitation avoidance, sample precipitation at relevant route locations and expected travel times, then prefer the candidate with lower precipitation exposure while enforcing the configured detour limit. Finally, construct a `RouteProposal` containing the selected route, optional place, extra travel time, confirmation text, and warnings.

Expected complexity. Let r be the number of route candidates and w be the number of weather samples evaluated across those candidates. Candidate filtering and ranking are $O(r \log r)$; weather evaluation is $O(rw)$, so the overall local selection cost is $O(r \log r + rw)$. For the expected small number of provider candidates, provider/network latency dominates. At 100× the candidate count, local work grows approximately linearly in r and w ; therefore candidate counts should be bounded and provider requests should have explicit timeouts.

Why this approach. A filter-then-rank strategy makes hard acceptance requirements explicit before preference scoring. A single weighted score without hard filtering could select a route that violates the 10-minute detour requirement. It also keeps weather as a preference rather than claiming that a route is guaranteed to be dry, matching DO's weather semantics.

Edge cases. No route candidates → `no_match`; no place within 600 seconds → alternate-place prompt; weather unavailable/stale → do not claim rain avoidance and continue with a warning; stale `tripRevision` → ignore the result; equal scores → deterministic tie-break by lower `extraSeconds`, then shorter duration; empty constraints → return the current/best normal route; provider quota/timeout → preserve current guidance

AI interpretation is treated as an input to validation; C3, not the provider, owns route selection.

C2/C3 Constraint-Aware Routing Flow

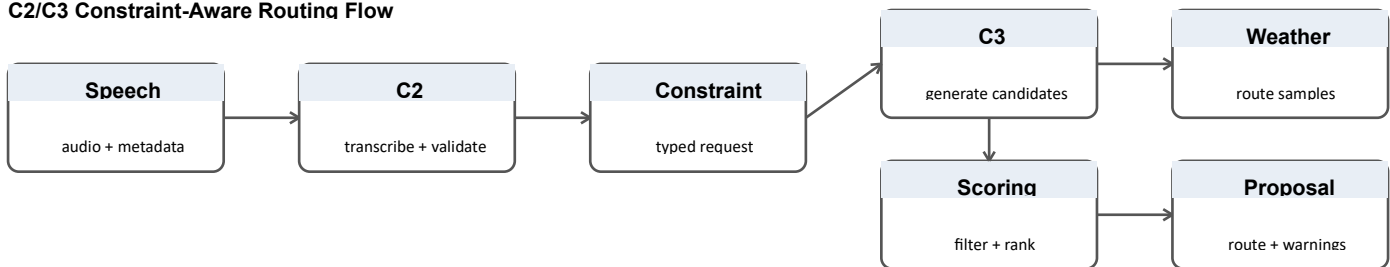


Figure 2. C2 validation and C3 route-selection flow.

Performance requirement. AC-01.1 measures from the end of the driver's spoken "okay" until the approved route is active in the map and turn-by-turn guidance. Proposal generation should therefore occur before confirmation whenever possible; after approval, C1/C2/C3 must perform only the necessary validation/refresh and activation work inside the 3-second budget.

4. Build-versus-Reuse Decisions

The guiding rule is to build Veer-specific behavior and reuse mature infrastructure for generic capabilities. Authentication, speech recognition, mapping, weather retrieval, and standard sorting/parsing are not hand-rolled.

Piece	Build / Reuse	Library/service	License / terms	Reason
Constraint validation	Build	Veer C2	Team code	Specific to Veer's supported constraint types and safety rules.
Route selection/scoring	Build	Veer C3	Team code	Core product behavior; must enforce Veer's detour/weather rules.
Speech transcription + interpretation	Reuse	OpenAI API	Proprietary service terms	Mature speech/language capability; avoids building a speech model.
Maps/routes/places	Reuse	Google Maps Platform	Proprietary Google terms	Building road graphs, place search, and route geometry is outside project scope.
Weather observation s/forecast	Reuse	Open-Meteo API	Service/data terms	Provides location/time weather data needed for precipitation evaluation.
HTTP/API server	Reuse	Node.js + Express	Node.js: MIT; Express: MIT	Mature backend tooling and avoids writing an HTTP server from scratch.
Relational data storage	Reuse	PostgreSQL	PostgreSQL License	Stable relational model and indexing for retained usage metadata.
Authentication/HTTP S primitives	Reuse	Platform/cloud security facilities	Provider terms	Security-sensitive primitives should not be custom implemented.

Maturity, licensing, performance, and fit check. Generic infrastructure is reused only when it is mature enough for a student project, has terms compatible with the project, meets the expected latency/throughput requirements, and directly fits the required interface. Provider retention and navigation/display terms remain deployment gates. In particular, the AI provider must support the D0 no-raw-audio-retention requirement before it is accepted as E2.

5. API Contract

D1 preserves the D0 interface IDs while making the C2/C3 contracts implementation-ready. All network requests include `requestId:string`. Errors use the D0 shape: `Error={code:string, message:string, retryable:boolean}`. A failed request never silently applies a route.

5.1 C1 → C2: I2 Speech Understanding

Field	Type / requirement	Valid range / meaning
<code>requestId</code>	string, required	Non-empty unique request identifier.
<code>tripId</code>	string, required	Current trip identifier.
<code>utteranceId</code>	string, required	Unique within trip/request lifecycle.
<code>control</code>	enum, required	audio or cancel.
<code>audio</code>	binary, conditional	Required for <code>control=audio</code> ; transient only.

audioMetadata	AudioMetadata, required	sampleRateHz/channels positive; utteranceEndedAt is ISO 8601 UTC.
response.constraint	Constraint?, optional	Required for a valid propose result; otherwise absent.
response.confidence	number, required when interpretation succeeds	0–1 inclusive.

Errors: `invalid_audio` (400, malformed/unsupported audio); `unsupported_constraint` (422, provider produced a type outside the allow-list); `unclear_speech` (422, interpretation cannot be safely validated); `quota` (429, provider quota exhausted); `provider_timeout` (504, provider exceeded deadline). For unclear speech or invalid interpretation, C1 asks for a repeat within 2 seconds and leaves the route unchanged.

5.2 C1 → C3: I4 Route Planning

Field	Type / requirement	Valid range / meaning
requestId	string, required	Unique request identifier.
tripId	string, required	Current trip identifier.
location	LocationFix, required	Current WGS84 position and capture timestamp.
destination	LatLng, required	Valid WGS84 latitude/longitude.
currentRoute	RoutePlan?, optional	Active route if one exists.
constraints	Constraint[], required	Zero or more validated active constraints.
tripRevision	integer, required	Non-negative current revision; stale responses are ignored.
proposal	RouteProposal?, conditional	Present on successful proposal; otherwise absent.

Errors: `no_match` (404/422, no route/place satisfies the requested constraint); `weather_unavailable` (503, weather avoidance cannot be verified); `quota` (429, provider budget/quota); `timeout` (504, provider deadline exceeded); `stale_revision` (409, response corresponds to an old trip revision). All errors preserve the active route.

5.3 C3 → E3/E4: Provider Contracts

I5 sends origin, destination, waypoints, optional place query, `avoidHighways`, `avoidTolls`, and alternatives to the Google Maps platform over HTTPS. The normalized result is `RoutePlan[]` plus `Place[]` and attribution. I6 sends location/time samples to the weather provider and receives `WeatherSample[]` plus `fetchAt`. C3 owns provider timeouts, quota handling, normalization, and the decision not to claim rain avoidance when weather coverage is unavailable.

5.4 Example request and response

POST /api/v1/trips/trip-17/route-proposals Content-Type: application/json

```
{
  "requestId": "req-18",
  "tripId": "trip-17",
  "location": {"lat": 39.1031, "lon": -84.5120},
  "destination": {"lat": 39.9612, "lon": -82.9988},
  "currentRoute": null,
  "constraints": [
    {
      "type": "add_waypoint",
      "value": "McDonald's",
      "maxDetourSeconds": 600
    }
  ]
}
```

```

    }
  ],
  "tripRevision": 4
}
{
  "proposal": {
    "tripRevision": 4,
    "selectedPlace": {
      "placeId": "place-123",
      "name": "McDonald's",
      "location": {"lat": 39.18, "lon": -84.47}
    },
    "extraSeconds": 420,
    "confirmationText": "McDonald's adds about seven minutes.",
    "warnings": []
  }
}

```

Versioning. External and internal HTTP endpoints use an explicit `/api/v1/` version. Changing required inputs, removing an endpoint, changing the meaning/type of an existing response field, or changing behavior in a way that breaks existing clients is a breaking change and requires a new major API version. Additive optional fields are non-breaking within a major version.

6. Technology Choices with Justification

The D0 architecture is a mobile client plus a shared backend with layered logical components rather than independently deployed microservices. D1 keeps that architecture and selects a practical implementation stack. The choices below are evaluated against team skill fit, licensing, community support, performance, and cost/hosting.

6.1 Mobile platform — Android/Kotlin

Android with Kotlin is selected because a mounted-phone first release can target a single platform while keeping microphone, location, speaker, background execution, and map integration under one mobile runtime. Kotlin is a mature, widely supported language with strong Android tooling and community documentation. The Android SDK and Kotlin ecosystem have established licensing/usage terms appropriate for student development. Performance is sufficient for local audio buffering, UI updates, location callbacks, and speech streaming without requiring custom vehicle hardware. Development can be performed on existing team computers and test phones, keeping hosting and hardware cost low. iOS/Swift was considered, but Android/Kotlin is preferred for the first release because it reduces the initial platform scope.

6.2 Backend — Node.js/TypeScript with Express

Node.js with TypeScript and Express is selected because the backend is primarily an integration service: it validates structured AI output, calls routing/weather APIs, manages trip revisions, and returns JSON contracts. TypeScript provides static types that match the D0 records and API contracts, while Node.js and Express have large communities and mature documentation. Node.js is open source under the MIT license and Express is MIT licensed. The event-driven runtime is appropriate for I/O-heavy provider calls and can meet Veer's short response budgets when provider calls are bounded by timeouts. Hosting a small stateless backend is inexpensive and can begin on a free/low-cost tier. Python/FastAPI was considered as an alternative; it is also capable, but TypeScript provides a direct type match between the backend contracts and the mobile/web-facing JSON.

6.3 Retained data — PostgreSQL

PostgreSQL is selected for retained usage metadata because C4 needs reliable trip IDs, provider events, deduplication identifiers, and summaries. The team can use standard SQL concepts and PostgreSQL's large community/documentation ecosystem. PostgreSQL uses the PostgreSQL License, which is permissive for the project. Indexes on `tripId/eventId` and transactional writes provide more than enough performance for the expected development/demo workload; the database is not on the critical speech-to-route computation path. A small hosted PostgreSQL instance is inexpensive, while local PostgreSQL can be used during development. MongoDB was considered, but PostgreSQL better matches the stable relationships and deduplication requirements of usage events.

6.4 AI provider — OpenAI API

An OpenAI API integration is selected as the initial E2 implementation because Veer needs transcription plus structured interpretation of spoken routing requests without training or hosting a speech model. The service has mature developer documentation and community examples, and the team can isolate provider-specific code behind I3 so the rest of C2 remains provider-independent. The service is proprietary and therefore is governed by provider terms rather than an open-source license; the implementation must verify that the selected configuration meets Veer's no-retention requirement before deployment. Latency must be measured because AC-01.4 requires a repeat prompt within 2 seconds after unparseable speech, while the final 3-second route-activation budget is measured after approval. On-device transcription was considered because it improves privacy and removes a network dependency, but hosted inference is selected initially for implementation simplicity and model capability.

6.5 Maps — Google Maps Platform

Google Maps Platform is retained from D0 because the project specifically requires route and place data, and reproducing a road network, routing engine, place database, and map attribution system would consume the project's development time. The platform has mature documentation and broad community support. Its APIs are proprietary and must be used under Google's current service terms and quota rules. Network latency is the principal performance risk, so C3 will request only the candidate routes and place information required for the active constraint and enforce bounded timeouts. Using an open-source routing stack such as OpenStreetMap-based self-hosted routing was considered, but it would add deployment and data-maintenance work and is less aligned with the D0 dependency.

6.6 Weather — Open-Meteo

Open-Meteo is selected as the initial E4 implementation because Veer needs location-indexed precipitation observations/forecasts rather than a full weather application. Its API is simple to integrate from the backend and avoids maintaining a weather-data pipeline. Community documentation and a straightforward HTTP interface reduce implementation effort. The service/data terms must be checked for the exact deployment and any attribution requirements before release. Performance is expected to be adequate because C3 can request only route-relevant sample points and cache nothing by default. A commercial weather API was considered, but Open-Meteo is a lower-cost first-release choice while the team validates the weather-avoidance behavior.

6.7 Hosting — Render with managed PostgreSQL

Render is selected for the first shared backend deployment because the project needs a simple hosted HTTPS endpoint rather than a distributed cloud architecture. The team can deploy a Node.js service and managed PostgreSQL without managing virtual machines, which is a good fit for a student team. The platform is proprietary but exposes a conventional deployment model and has community documentation. A small service is sufficient for the expected development/demo load, and the team can control spending with provider budgets and C4 usage tracking. AWS was considered, but its broader service surface would add configuration and operational overhead that does not improve the first-release product enough to justify it.

6.8 Cost and operational limits

The project remains team-funded. C4 records AI, Maps, and weather request attempts per trip, including retries, so provider consumption is visible. The first release uses the smallest practical hosted backend/database configuration and prioritizes free tiers, credits, or development quotas where available. No provider is permitted to silently retain raw audio, and no provider failure may silently change the route. Spending limits are treated as an operational requirement rather than a routing behavior.

D1 Traceability and Implementation Checklist

Requirement / decision	D1 evidence
US-01 / AC-01.1	C3 route proposal, 600-second detour bound, explicit approval, and 3-second activation budget.
US-02	C3 weather sampling and precipitation-aware candidate ranking; weather is a preference, not a guarantee.
US-03	C4 remains D2, but C2/C3 emit typed UsageEvents for every provider attempt.
US-05 / D07	C2 returns a proposal only; C1 owns spoken confirmation and final route application.
AC-01.2	C2/provider selection requires transient audio processing and no raw-audio retention.
AC-01.3	No place/route within the detour bound leaves the current route unchanged and produces an alternate-place recovery prompt.
AC-01.4	Unparseable speech becomes an error/unknown result and triggers a repeat prompt within 2 seconds.
D04	Speech is represented as typed routing constraints rather than free-form instructions to the routing system.
D05	Google Maps supplies route/place data while weather remains a separate C3 input.
D08	Optional replanning failures preserve the active route and continue existing guidance.

Implementation boundary. C2 owns interpretation validation and C3 owns route selection. C1 remains the only component allowed to activate the approved route. C3 may propose a route but never mutates the live navigation state. C4 receives only provider-consumption metadata. This preserves the D0 responsibility boundaries while providing enough detail for implementation.

Items to verify during implementation. Provider-specific audio encoding/wire details; actual provider retention settings; current Maps navigation/display terms; exact precipitation threshold, weather freshness window, and maximum weather detour; mobile background-execution behavior; and actual spending caps. These are implementation verification items, not changes to the D0 component responsibilities.